



TECNICO EN CIBERSEGURIDAD

Participante: Luis Mario Reynoso

Técnico: Ciberseguridad

Nombre del Módulo: CRIPTOGRAFÍA Y CIFRADO DE DATOS

Facilitador: Kevin Feliz Henríquez

Centro: Cyberclass

Nombre Actividad: Criptografía Polialfabética

Descripción: En esta guía aprendimos el funcionamiento del Cifrado Vigenère, uno de los algoritmos clásicos más importantes de la criptografía. Comprendimos cómo una clave puede generar múltiples desplazamientos dentro del mensaje, haciendo el cifrado considerablemente más robusto que César y ROT13. Además, implementamos funciones de cifrado y descifrado en Python, utilizamos operaciones matemáticas modulares y construimos una aplicación interactiva completa

1. ¿Qué hace el programa?

El programa CryptoVigenere Suite sirve para cifrar y descifrar mensajes utilizando el cifrado Vigenère. Este método usa una clave escrita por el usuario para generar diferentes desplazamientos sobre las letras del mensaje. A diferencia del cifrado César o ROT13, Vigenère no usa un único desplazamiento, sino varios desplazamientos según la clave.

Elemento	Descripción	Ejemplo
Mensaje original	Texto que el usuario quiere proteger.	HOLA
Clave	Palabra secreta usada para cifrar o descifrar.	KFH
Mensaje cifrado	Resultado generado por el algoritmo.	RTSK

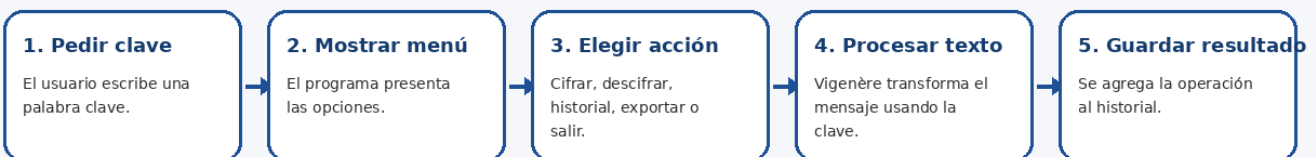
2. ¿Para qué sirve digitar una clave?

La clave funciona como una contraseña del cifrado. Cuando una persona cifra un mensaje, debe usar una clave. Luego, la persona que quiera descifrar el mensaje necesita escribir la misma clave. Si usa otra clave, el resultado no será correcto.

- La clave indica cuánto se mueve cada letra del mensaje.
- La clave se repite automáticamente hasta igualar la longitud del texto.
- La misma clave debe usarse para descifrar correctamente.

3. Flujo general del programa

Flujo general del programa



4. Definición de cada función

A continuación se define cada parte importante del archivo app.py. Cada sección incluye el propósito, la explicación y una captura representativa del código en Visual Studio Code.

4.1 Variable global: ALFABETO

Definición: Define las letras válidas que el programa usará para cifrar y descifrar. Sirve como base para validar caracteres y convertir letras en posiciones numéricas.

Puntos importantes:

- Contiene las letras de la A hasta la Z en mayúscula.
- Permite comprobar si un carácter es una letra o si debe conservarse igual.
- Se usa dentro de limpiar_clave(), cifrar_vigenere() y descifrar_vigenere().

Captura del código:

```
app.py > principal
1  # GUÍA #3 - Cifrado y Descifrado Vigenère
2  # CryptoVigenere Suite
3
4  ALFABETO = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
5
6
```

4.2 Función limpiar_clave(clave)

Definición: Limpia la clave escrita por el usuario y deja solamente letras. Esto evita errores cuando la clave contiene espacios, números o símbolos.

Puntos importantes:

- Convierte cada carácter de la clave a mayúscula.
- Verifica si el carácter pertenece al alfabeto.
- Devuelve una clave válida para el algoritmo Vigenère.

Captura del código:

```
6
7  def limpiar_clave(clave):
8      """
9      Deja la clave solamente con letras A-Z.
10     Ejemplo: 'K F H 123' -> 'KFH'
11     """
12     clave_limpia = ""
13
14     for letra in clave:
15         letra = letra.upper()
16
17         if letra in ALFABETO:
18             clave_limpia += letra
19
20     return clave_limpia
21
```

5.3 Función cifrar_vigenere(texto, clave)

Definición: Recibe un mensaje normal y una clave, luego transforma el mensaje en texto cifrado usando el algoritmo Vigenère.

Puntos importantes:

- Limpia la clave antes de usarla.
- Solo cifra letras; los espacios, números y signos se mantienen igual.
- Mantiene el formato original: mayúsculas siguen mayúsculas y minúsculas siguen minúsculas.
- Aplica la fórmula: texto + clave módulo 26.

Captura del código:

```
def cifrar_vigenere(texto, clave):
    resultado = ""
    clave = limpiar_clave(clave)

    if clave == "":
        return None

    indice_clave = 0

    for caracter in texto:
        letra_mayuscula = caracter.upper()

        if letra_mayuscula in ALFABETO:
            valor_texto = ord(letra_mayuscula) - 65
            valor_clave = ord(clave[indice_clave % len(clave)]) - 65

            cifrado = (valor_texto + valor_clave) % 26

            if caracter.isupper():
                resultado += chr(cifrado + 65)
            else:
                resultado += chr(cifrado + 97)

            indice_clave += 1

        else:
            resultado += caracter

    return resultado
```

5.4 Función descifrar_vigenere(texto, clave)

Definición: Hace el proceso contrario al cifrado. Recibe un mensaje cifrado y la misma clave usada para cifrarlo, y recupera el mensaje original.

Puntos importantes:

- Usa la misma clave que se utilizó en el cifrado.
- Resta el valor de la clave al valor del texto cifrado.
- Conserva los espacios, números y signos sin cambiarlos.
- Aplica la fórmula: texto - clave módulo 26.

Captura del código:

```
def descifrar_vigenere(texto, clave):
    resultado = ""
    clave = limpiar_clave(clave)

    if clave == "":
        return None

    indice_clave = 0

    for caracter in texto:
        letra_mayuscula = caracter.upper()

        if letra_mayuscula in ALFABETO:
            valor_texto = ord(letra_mayuscula) - 65
            valor_clave = ord(clave[indice_clave % len(clave)]) - 65

            descifrado = (valor_texto - valor_clave) % 26

            if caracter.isupper():
                resultado += chr(descifrado + 65)
            else:
                resultado += chr(descifrado + 97)

            indice_clave += 1

        else:
            resultado += caracter

    return resultado
```

5.5 Función mostrar_menu()

Definición: Muestra en pantalla las opciones disponibles para que el usuario pueda interactuar con el programa.

Puntos importantes:

- Presenta las opciones principales del sistema.
- Permite cifrar, descifrar, ver historial, cambiar clave, exportar y ver estadísticas.
- Se ejecuta en cada vuelta del ciclo principal.

Captura del código:

```
def mostrar_menu():
    print("\n===== CRYPTOVIGENERE SUITE =====")
    print("1. Cifrar mensaje")
    print("2. Descifrar mensaje")
    print("3. Ver historial")
    print("4. Cambiar clave")
    print("5. Exportar historial")
    print("6. Ver estadísticas")
    print("7. Salir")
```

5.6 Función mostrar_historial(historial)

Definición: Muestra todos los mensajes cifrados o descifrados durante la ejecución del programa.

Puntos importantes:

- Recibe una lista llamada historial.
- Si no hay registros, muestra un aviso.
- Si hay registros, imprime operación, texto, clave y resultado.

Captura del código:

```
def mostrar_historial(historial):
    print("\n===== HISTORIAL =====")

    if len(historial) == 0:
        print("No existen registros.")
    else:
        for registro in historial:
            print(f"Operación : {registro['operacion']}")
            print(f"Texto      : {registro['texto']}")
            print(f"Clave       : {registro['clave']}")
            print(f"Resultado   : {registro['resultado']}")
            print("-----")
```

5.7 Función exportar_historial(historial)

Definición: Guarda el historial del programa en un archivo de texto llamado historial_vigenere.txt.

Puntos importantes:

- Verifica si existe historial antes de exportar.
- Crea un archivo TXT en la misma carpeta del proyecto.
- Guarda cada operación con su texto, clave y resultado.

Captura del código:

```
def exportar_historial(historial):
    if len(historial) == 0:
        print("\nNo hay historial para exportar.")
        return

    archivo = open("historial_vigenere.txt", "w", encoding="utf-8")

    archivo.write("==== HISTORIAL VIGENERE ==== \n\n")

    for registro in historial:
        archivo.write(f"Operación : {registro['operacion']} \n")
        archivo.write(f"Texto      : {registro['texto']} \n")
        archivo.write(f"Clave   : {registro['clave']} \n")
        archivo.write(f"Resultado : {registro['resultado']} \n")
        archivo.write("----- \n")

    archivo.close()

    print("\nHistorial exportado correctamente en historial_vigenere.txt")
```

5.8 Función mostrar_estadisticas(contador_cifrados, contador_descifrados)

Definición: Presenta un conteo de cuántas operaciones se han realizado durante el uso del programa.

Puntos importantes:

- Muestra la cantidad de cifrados realizados.
- Muestra la cantidad de descifrados realizados.
- Calcula el total sumando ambos contadores.

Captura del código:

```
def mostrar_estadisticas(contador_cifrados, contador_descifrados):
    total = contador_cifrados + contador_descifrados

    print("\n==== ESTADÍSTICAS ====")
    print(f"Cifrados realizados      : {contador_cifrados}")
    print(f"Descifrados realizados   : {contador_descifrados}")
    print(f"Total de operaciones     : {total}")
```

5.9 Función pedir_clave()

Definición: Solicita una clave al usuario y no permite continuar hasta que la clave tenga al menos una letra válida.

Puntos importantes:

- Pide al usuario que digite una clave.
- Llama a limpiar_clave() para validar la entrada.
- Evita que el programa falle por una clave vacía.

Captura del código:

```
def pedir_clave():  
    while True:  
        clave = input("Digite la clave: ")  
        clave_limpia = limpiar_clave(clave)  
  
        if clave_limpia != "":  
            return clave  
  
    print("La clave debe contener al menos una letra.")
```


5.10 Función principal()

Definición: Controla todo el funcionamiento del programa. Aquí se crea el historial, los contadores y se ejecuta el menú interactivo.

Puntos importantes:

- Crea una lista vacía para guardar el historial.
- Define los contadores de cifrado y descifrado.
- Pide la clave inicial.
- Usa un ciclo while True para mantener el programa abierto hasta elegir salir.
- Llama a las demás funciones según la opción seleccionada.

Captura del código:

```
def principal():
    historial = []

    contador_cifrados = 0
    contador_descifrados = 0

    clave_actual = pedir_clave()

    while True:
        mostrar_menu()

        opcion = input("Seleccione una opción: ")

        if opcion == "1":
            mensaje = input("\nDigite el mensaje: ")

            resultado = cifrar_vigenere(mensaje, clave_actual)

            print("\nMensaje cifrado:")
            print(resultado)

            historial.append({
                "operacion": "Cifrado",
                "texto": mensaje,
                "clave": clave_actual,
                "resultado": resultado
            })
```

```

61 def principal():
62     contador_cifrados += 1
63
64     elif opcion == "2":
65         mensaje = input("\nDigite el mensaje cifrado: ")
66
67         resultado = descifrar_vigenere(mensaje, clave_actual)
68
69         print("\nMensaje descifrado:")
70         print(resultado)
71
72         historial.append({
73             "operacion": "Descifrado",
74             "texto": mensaje,
75             "clave": clave_actual,
76             "resultado": resultado
77         })
78         contador_descifrados += 1
79
80     elif opcion == "3":
81         mostrar_historial(historial)
82
83     elif opcion == "4":
84         clave_actual = pedir_clave()
85         print("\nClave cambiada correctamente.")
86
87     elif opcion == "5":
88         exportar_historial(historial)
89
90     elif opcion == "6":
91         mostrar_estadisticas(contador_cifrados, contador_descifrados)
92
93     elif opcion == "7":
94         print("\nFinalizando programa...")
95         break
96
97     else:
98         print("\nOpción inválida. Intente nuevamente.")
99
100 principal()
```

8. Conclusión

El proyecto CryptoVigenere Suite demuestra cómo aplicar criptografía polialfabética usando Python. El programa permite cifrar, descifrar, cambiar claves, guardar historial, exportar información y ver estadísticas. También enseña el uso de funciones, ciclos, condicionales, listas, diccionarios, manejo de archivos y operaciones matemáticas modulares.

